TD - GAN

Partie 1 — La convolution transposée

1. 1. Charger l'image einstein.png fournie avec le sujet, la convertir en niveaux de gris, puis en tenseur de forme (1, 1, H, W), à valeurs dans [0, 1].

1. 2. Lui appliquer nn.ConvTranspose2d(1, 1, kernel_size=3, stride=2, bias=False) avec tous les poids à 1/4, et afficher le résultat. L'image est agrandie d'un facteur 2 (la sortie fait exactement 2H+1 pixels de côté), mais on observe un artefact en damier : le noyau n'étant pas un multiple du stride, certains pixels reçoivent plus de contributions que leurs voisins.

1. 3. Refaire avec kernel_size=4, padding=1, en conservant stride=2 (poids à 1/4). L'image est agrandie sans damier et la sortie fait exactement 2H : chaque pixel reçoit le même nombre de contributions.

1. 4. Refaire avec un sur-échantillonnage simple suivi d'une convolution classique : nn.Upsample(scale_factor=2) puis nn.Conv2d(1, 1, 3, padding=1, bias=False), poids à 1/9. L'image est agrandie et légèrement lissée, sans damier.

Partie 2 — DCGAN sur MNIST

Implémenter un DCGAN avec PyTorch et l'entraîner sur MNIST (images 32×32, normalisées dans [-1, 1]).

Générateur (bruit z de dimension 100) :
- ConvTranspose2d(100, 128, 4, 1, 0) + BatchNorm + ReLU -> 128×4×4
- ConvTranspose2d(128, 64, 4, 2, 1) + BatchNorm + ReLU -> 64×8×8
- ConvTranspose2d(64, 32, 4, 2, 1) + BatchNorm + ReLU -> 32×16×16
- ConvTranspose2d(32, 1, 4, 2, 1) + Tanh -> 1×32×32

Discriminateur :
- Conv2d(1, 32, 4, 2, 1) + LeakyReLU(0,2) -> 32×16×16
- Conv2d(32, 64, 4, 2, 1) + BatchNorm + LeakyReLU(0,2) -> 64×8×8
- Conv2d(64, 128, 4, 2, 1) + BatchNorm + LeakyReLU(0,2) -> 128×4×4
- Conv2d(128, 1, 4, 1, 0) -> 1 logit

Hyperparamètres : batch 128 ; Adam, lr = 2e-4, betas = (0,5 ; 0,999) pour G et D ; poids initialisés selon N(0 ; 0,02) ; perte BCEWithLogitsLoss ; 5 époques sur les 60 000 images (sous-ensemble de 20 000 images si le CPU est lent).

Apprentissage, à chaque itération :
1. Mise à jour de D
   - Tirer un lot de bruits z ~ N(0, I) de taille (128, 100) et générer fake = G(z). N(0, I) est la loi normale multivariée : chacune des 100 composantes d'un z suit N(0, 1), indépendamment des autres. Un z neuf est tiré à chaque itération, sans quoi D apprendrait par cœur ces 128 fausses images au lieu d'apprendre à les reconnaître.
   - Faire juger les deux lots par D : logits_vrais = D(x) sur un lot de vraies images, logits_faux = D(fake.detach()) sur les fausses. D renvoie des logits et non des probabilités : la sigmoïde est incluse dans BCEWithLogitsLoss, ce qui est numériquement plus stable.

- Perte de D = BCE(logits_vrais, 1) + BCE(logits_faux, 0) : D apprend à répondre 1 sur les vraies images et 0 sur les fausses. Repère utile pour lire les courbes : un discriminateur qui répondrait 0,5 partout, donc au hasard, obtiendrait $2 \times \log 2 \approx 1{,}386$.
- Appliquer la mise à jour, dans cet ordre :
  • opt_D.zero_grad() remet à zéro les gradients de l'itération précédente : PyTorch les additionne au lieu de les remplacer ;
  • loss_D.backward() calcule par rétropropagation le gradient de la perte par rapport à chaque poids de D, sans encore rien modifier ;
  • opt_D.step() met à jour ces poids dans la direction qui fait baisser la perte, selon la règle d'Adam. Seul D change, car opt_D ne connaît que D.parameters().
- C'est ici que le fake.detach() de la deuxième puce prend son sens : il coupe le graphe, donc la rétropropagation s'arrête aux images et ne traverse pas G. Sans lui, le graphe de G serait parcouru pour rien, puis libéré — et l'étape 2, qui réutilise fake, ne pourrait plus s'en servir.

2. Mise à jour de G
- Refaire juger les mêmes images par D, cette fois sans detach() : logits = D(fake). Le gradient pourra ainsi traverser D et atteindre G — non pour corriger D, mais parce que c'est le seul chemin par lequel G apprend en quoi ses images ne convainquent pas. D sert de juge dérivable.
- Perte de G = BCE(logits, 1) : la cible est 1, alors que ces mêmes images avaient la cible 0 à l'étape 1 — c'est tout le jeu adverse. G est récompensé quand D prend ses images pour des vraies. Cette perte « non saturante » donne des gradients plus forts en début d'entraînement que minimiser $\log(1 - D(G(z)))$.
- Appliquer la mise à jour, dans le même ordre qu'à l'étape 1 :
  • opt_G.zero_grad() ;
  • loss_G.backward() : le gradient traverse D, puis entre dans G ;
  • opt_G.step() : seuls les poids de G changent, même si le gradient a traversé D, car opt_G ne connaît que G.parameters().
- Au passage, cette rétropropagation a laissé des gradients dans D. Ils sont sans effet : opt_G n'y touche pas, et le opt_D.zero_grad() de l'itération suivante les efface.

Suivi :
- tirer une fois pour toutes un bruit fixe fixed_z de 64 vecteurs : en réutilisant les mêmes z à chaque époque, les différences entre deux grilles ne viennent que de l'apprentissage de G, jamais du tirage ;
- à la fin de chaque époque, afficher G(fixed_z) en grille 8×8 avec torchvision.utils.make_grid, en basculant G en mode eval puis en le remettant en train. Ce basculement est imposé par les BatchNorm : en mode train elles normalisent avec les statistiques du lot courant, si bien que l'image obtenue pour un z dépendrait des 63 autres z de la grille ;
- enregistrer les pertes de D et de G à chaque itération et les tracer.

Sur les 60 000 images (468 itérations par époque), on observe des traits flous dès la 1re époque et des chiffres lisibles vers la 3e. Avec le sous-ensemble de 20 000 images ces repères arrivent plus tard : la 3e époque n'y représente que 468 itérations au total, soit l'équivalent de la 1re époque sur le jeu complet. Les pertes oscillent sans tendre vers 0 : elles n'indiquent pas la qualité des images.

Partie 3 — Les limites des GAN

Relancer le DCGAN de la Partie 2 (2 à 3 époques) en changeant uniquement les réglages indiqués.

3.1 Entraînement instable
- Réglages : lr de G et de D = 2e-3, betas = (0,9 ; 0,999).

- Observation : les pertes oscillent fortement et les images changent brutalement d'une époque à l'autre, jusqu'à redevenir du bruit.
- Variante : lr de D = 1e-3, lr de G = 5e-5. Le discriminateur gagne : D(G(z)) tombe à 0, la perte de G explose et les images cessent de progresser.

3.2 Mode collapse
- Réglages : pas de BatchNorm dans G, lr de G = 1e-3, 5 pas de G pour 1 pas de D.
- Observation : des bruits différents donnent des images presque identiques, et un ou deux chiffres dominent la grille.

3.3 Mauvaise qualité
- Réglages : 8 filtres de base au lieu de 32, dans G comme dans D, convolutions transposées à noyau 3 (stride 2, padding 1, output_padding 1).
- Observation : chiffres flous et cassés, avec le damier vu en Partie 1. En une seule époque le damier est déjà très marqué, mais les chiffres ne sont pas encore formés : il faut les trois époques pour voir le défaut de qualité lui-même.
- Attention : les 8 filtres s'appliquent aux deux réseaux. En ne rétrécissant que G, le discriminateur reste quatre fois plus large, gagne aussitôt (D(G(z)) ≈ 0,006) et l'on n'obtient que du bruit : le défaut de qualité disparaît alors derrière un défaut d'équilibre.

Remèdes à tester ensuite : cible 0,9 pour les vraies images (label smoothing), normalisation spectrale sur D, ou perte WGAN-GP. Attention : appliqués au réglage de 3.1, les deux premiers atténuent les symptômes sans redresser l'entraînement — le label smoothing est le plus efficace des deux. La cause reste le pas d'optimisation : le vrai remède est de revenir à lr = 2e-4 et betas = (0,5 ; 0,999).

Partie 4 — Cas pratique : un mini-ProGAN

Entraîner un GAN qui grandit progressivement sur MNIST : 4×4 -> 8×8 -> 16×16 -> 32×32.

1. **Données** : à chaque phase, réduire les images 32×32 à la résolution courante (F.avg_pool2d).

2. **Générateur** : un bloc initial ConvTranspose2d(100, 128, 4) -> 128×4×4, puis un bloc par résolution supplémentaire : Upsample(×2) + Conv2d(3×3) + LeakyReLU(0,2). Une couche toImage (Conv2d 1×1 vers 1 canal) par résolution. Nombre de filtres par résolution : 128 en 4×4, 128 en 8×8, 64 en 16×16, 32 en 32×32.

3. **Discriminateur** : symétrique, avec une couche fromImage (1×1) par résolution, puis des blocs Conv2d(3×3) + LeakyReLU(0,2) + AvgPool2d(2).

4. **Fondu** : pendant la première moitié de chaque nouvelle phase, la sortie de G vaut $(1 - \alpha) \cdot$ upsample(toImage ancien) $+ \alpha \cdot$ toImage nouveau, avec α qui passe linéairement de 0 à 1. Même mélange à l'entrée de D, mais en sens inverse : $\alpha \cdot$ bloc(fromImage nouveau(x)) $+ (1 - \alpha) \cdot$ fromImage ancien(avgpool(x)), les deux termes étant pris à la résolution précédente.

5. **Hyperparamètres** : même perte et même optimiseur qu'en Partie 2 ; 40 000 images par phase (moitié fondu, moitié stabilisation). 10 000 images ne suffisent pas : la phase 32×32 n'aurait que 39 itérations de stabilisation, le discriminateur finit par y écraser G et les images redeviennent noires. En revanche, garder ici l'initialisation par défaut de PyTorch et non le N(0 ; 0,02) de la Partie 2 : ce

générateur n'a pas de BatchNorm, et des poids aussi petits écrasent le signal couche après couche (G sort des images à ±0,003 et reste gris).

6. **Suivi** : afficher G(fixed_z) à la fin de chaque phase.

On observe d'abord des taches 4×4, puis la silhouette du chiffre, puis la finesse des traits : le réseau apprend la structure grossière avant les détails. L'entraînement est plus stable qu'avec le DCGAN, car chaque phase est un problème plus simple.